

INTRODUCTION

The pace at which computer systems change was, is, and continues to be overwhelming. From 1945, when the modern computer era began, until about 1985, computers were large and expensive. Moreover, for lack of a way to connect them, these computers operated independently from one another.

Starting in the mid-1980s, however, two advances in technology began to change that situation. The first was the development of powerful microprocessors. Initially, these were 8-bit machines, but soon 16-, 32-, and 64-bit CPUs became common. With multicore CPUs, we now are refacing the challenge of adapting and developing programs to exploit parallelism. In any case, the current generation of machines have the computing power of the mainframes deployed 30 or 40 years ago, but for 1/1000th of the price or less.

The second development was the invention of high-speed computer networks. **Local-area networks** or **LANs** allow thousands of machines within a building to be connected in such a way that small amounts of information can be transferred in a few microseconds or so. Larger amounts of data can be moved between machines at rates of billions of *bits per second* (**bps**). **Wide-area networks** or **WANs** allow hundreds of millions of machines all over the earth to be connected at speeds varying from tens of thousands to hundreds of millions bps.

Parallel to the development of increasingly powerful and networked machines, we have also been able to witness miniaturization of computer systems with perhaps the smartphone as the most impressive outcome. Packed with sensors, lots of memory, and a powerful CPU, these devices are nothing less than full-fledged computers. Of course, they also have networking capabilities. Along the same lines, so-called **plug computers** are finding their way to the

A version of this chapter has been published as “A Brief Introduction to Distributed Systems,” Computing, vol. 98(10):967-1009, 2016.

market. These small computers, often the size of a power adapter, can be plugged directly into an outlet and offer near-desktop performance.

The result of these technologies is that it is now not only feasible, but easy, to put together a computing system composed of a large numbers of networked computers, be they large or small. These computers are generally geographically dispersed, for which reason they are usually said to form a **distributed system**. The size of a distributed system may vary from a handful of devices, to millions of computers. The interconnection network may be wired, wireless, or a combination of both. Moreover, distributed systems are often highly dynamic, in the sense that computers can join and leave, with the topology and performance of the underlying network almost continuously changing.

In this chapter, we provide an initial exploration of distributed systems and their design goals, and follow that up by discussing some well-known types of systems.

1.1 What is a distributed system?

Various definitions of distributed systems have been given in the literature, none of them satisfactory, and none of them in agreement with any of the others. For our purposes it is sufficient to give a loose characterization:

A distributed system is a collection of autonomous computing elements that appears to its users as a single coherent system.

This definition refers to two characteristic features of distributed systems. The first one is that a distributed system is a collection of computing elements each being able to behave independently of each other. A computing element, which we will generally refer to as a **node**, can be either a hardware device or a software process. A second feature is that users (be they people or applications) believe they are dealing with a single system. This means that one way or another the autonomous nodes need to collaborate. How to establish this collaboration lies at the heart of developing distributed systems. Note that we are not making any assumptions concerning the type of nodes. In principle, even within a single system, they could range from high-performance mainframe computers to small devices in sensor networks. Likewise, we make no assumptions concerning the way that nodes are interconnected.

Characteristic 1: Collection of autonomous computing elements

Modern distributed systems can, and often will, consist of all kinds of nodes, ranging from very big high-performance computers to small plug computers or even smaller devices. A fundamental principle is that nodes can act independently from each other, although it should be obvious that if they ignore

each other, then there is no use in putting them into the same distributed system. In practice, nodes are programmed to achieve common goals, which are realized by exchanging messages with each other. A node reacts to incoming messages, which are then processed and, in turn, leading to further communication through message passing.

An important observation is that, as a consequence of dealing with independent nodes, each one will have its own notion of time. In other words, we cannot always assume that there is something like a **global clock**. This lack of a common reference of time leads to fundamental questions regarding the synchronization and coordination within a distributed system, which we will come to discuss extensively in Chapter 6. The fact that we are dealing with a *collection* of nodes implies that we may also need to manage the membership and organization of that collection. In other words, we may need to register which nodes may or may not belong to the system, and also provide each member with a list of nodes it can directly communicate with.

Managing **group membership** can be exceedingly difficult, if only for reasons of admission control. To explain, we make a distinction between open and closed groups. In an **open group**, any node is allowed to join the distributed system, effectively meaning that it can send messages to any other node in the system. In contrast, with a **closed group**, only the members of that group can communicate with each other and a separate mechanism is needed to let a node join or leave the group.

It is not difficult to see that admission control can be difficult. First, a mechanism is needed to authenticate a node, and as we shall see in Chapter 9, if not properly designed, managing authentication can easily create a scalability bottleneck. Second, each node must, in principle, check if it is indeed communicating with another group member and not, for example, with an intruder aiming to create havoc. Finally, considering that a member can easily communicate with nonmembers, if confidentiality is an issue in the communication within the distributed system, we may be facing trust issues.

Concerning the organization of the collection, practice shows that a distributed system is often organized as an **overlay network** [Tarkoma, 2010]. In this case, a node is typically a software process equipped with a list of other processes it can directly send messages to. It may also be the case that a neighbor needs to be first looked up. Message passing is then done through TCP/IP or UDP channels, but as we shall see in Chapter 4, higher-level facilities may be available as well. There are roughly two types of overlay networks:

Structured overlay: In this case, each node has a well-defined set of neighbors with whom it can communicate. For example, the nodes are organized in a tree or logical ring.

Unstructured overlay: In these overlays, each node has a number of references to randomly selected other nodes.

In any case, an overlay network should, in principle, always be **connected**, meaning that between any two nodes there is always a communication path allowing those nodes to route messages from one to the other. A well-known class of overlays is formed by **peer-to-peer (P2P) networks**. Examples of overlays will be discussed in detail in Chapter 2 and later chapters. It is important to realize that the organization of nodes requires special effort and that it is sometimes one of the more intricate parts of distributed-systems management.

Characteristic 2: Single coherent system

As mentioned, a distributed system should appear as a single coherent system. In some cases, researchers have even gone so far as to say that there should be a single-system view, meaning that end users should not even notice that they are dealing with the fact that processes, data, and control are dispersed across a computer network. Achieving a single-system view is often asking too much, for which reason, in our definition of a distributed system, we have opted for something weaker, namely that it *appears* to be coherent. Roughly speaking, a distributed system is coherent if it behaves according to the expectations of its users. More specifically, in a single coherent system the collection of nodes as a whole operates the same, no matter where, when, and how interaction between a user and the system takes place.

Offering a single coherent view is often challenging enough. For example, it requires that an end user would not be able to tell exactly on which computer a process is currently executing, or even perhaps that part of a task has been spawned off to another process executing somewhere else. Likewise, where data is stored should be of no concern, and neither should it matter that the system may be replicating data to enhance performance. This so-called **distribution transparency**, which we will discuss more extensively in Section 1.2, is an important design goal of distributed systems. In a sense, it is akin to the approach taken in many Unix-like operating systems in which resources are accessed through a unifying file-system interface, effectively hiding the differences between files, storage devices, and main memory, but also networks.

However, striving for a single coherent system introduces an important trade-off. As we cannot ignore the fact that a distributed system consists of multiple, networked nodes, it is inevitable that at any time only a part of the system fails. This means that unexpected behavior in which, for example, some applications may continue to execute successfully while others come to a grinding halt, is a reality that needs to be dealt with. Although partial failures are inherent to any complex system, in distributed systems they are particularly difficult to hide. It lead Turing-Award winner Leslie Lamport, to describe a distributed system as “[...] one in which the failure of a computer you didn’t even know existed can render your own computer unusable.”

Middleware and distributed systems

To assist the development of distributed applications, distributed systems are often organized to have a separate layer of software that is logically placed on top of the respective operating systems of the computers that are part of the system. This organization is shown in Figure 1.1, leading to what is known as **middleware** [Bernstein, 1996].

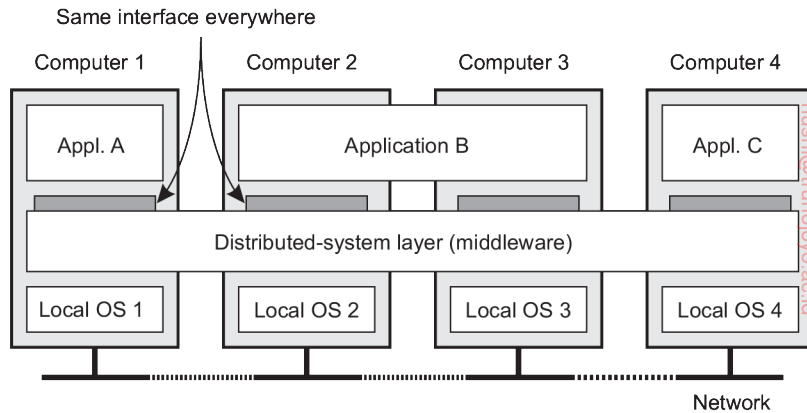


Figure 1.1: A distributed system organized in a middleware layer, which extends over multiple machines, offering each application the same interface.

Figure 1.1 shows four networked computers and three applications, of which application B is distributed across computers 2 and 3. Each application is offered the same interface. The distributed system provides the means for components of a single distributed application to communicate with each other, but also to let different applications communicate. At the same time, it hides, as best and reasonably as possible, the differences in hardware and operating systems from each application.

In a sense, middleware is the same to a distributed system as what an operating system is to a computer: a manager of resources offering its applications to efficiently share and deploy those resources across a network. Next to resource management, it offers services that can also be found in most operating systems, including:

- Facilities for interapplication communication.
- Security services.
- Accounting services.
- Masking of and recovery from failures.

The main difference with their operating-system equivalents, is that middleware services are offered in a networked environment. Note also that most services are useful to many applications. In this sense, middleware can

also be viewed as a container of commonly used components and functions that now no longer have to be implemented by applications separately. To further illustrate these points, let us briefly consider a few examples of typical middleware services.

Communication: A common communication service is the so-called **Remote Procedure Call (RPC)**. An RPC service, to which we return in Chapter 4, allows an application to invoke a function that is implemented and executed on a remote computer as if it was locally available. To this end, a developer need merely specify the function header expressed in a special programming language, from which the RPC subsystem can then generate the necessary code that establishes remote invocations.

Transactions: Many applications make use of multiple services that are distributed among several computers. Middleware generally offers special support for executing such services in an all-or-nothing fashion, commonly referred to as an atomic **transaction**. In this case, the application developer need only specify the remote services involved, and by following a standardized protocol, the middleware makes sure that every service is invoked, or none at all.

Service composition: It is becoming increasingly common to develop new applications by taking existing programs and gluing them together. This is notably the case for many Web-based applications, in particular those known as **Web services** [Alonso et al., 2004]. Web-based middleware can help by standardizing the way Web services are accessed and providing the means to generate their functions in a specific order. A simple example of how service composition is deployed is formed by **mashups**: Web pages that combine and aggregate data from different sources. Well-known mashups are those based on Google maps in which maps are enhanced with extra information such as trip planners or real-time weather forecasts.

Reliability: As a last example, there has been a wealth of research on providing enhanced functions for building reliable distributed applications. The Horus toolkit [van Renesse et al., 1994] allows a developer to build an application as a group of processes such that any message sent by one process is guaranteed to be received by all or no other process. As it turns out, such guarantees can greatly simplify developing distributed applications and are typically implemented as part of the middleware.

Note 1.1 (Historical note: The term middleware)

Although the term middleware became popular in the mid 1990s, it was most likely mentioned for the first time in a report on a NATO software engineering

conference, edited by Peter Naur and Brian Randell in October 1968 [Naur and Randell, 1968]. Indeed, middleware was placed precisely between applications and service routines (the equivalent of operating systems).

1.2 Design goals

Just because it is possible to build distributed systems does not necessarily mean that it is a good idea. In this section we discuss four important goals that should be met to make building a distributed system worth the effort. A distributed system should make resources easily accessible; it should hide the fact that resources are distributed across a network; it should be open; and it should be scalable.

Supporting resource sharing

An important goal of a distributed system is to make it easy for users (and applications) to access and share remote resources. Resources can be virtually anything, but typical examples include peripherals, storage facilities, data, files, services, and networks, to name just a few. There are many reasons for wanting to share resources. One obvious reason is that of economics. For example, it is cheaper to have a single high-end reliable storage facility be shared than having to buy and maintain storage for each user separately.

Connecting users and resources also makes it easier to collaborate and exchange information, as is illustrated by the success of the Internet with its simple protocols for exchanging files, mail, documents, audio, and video. The connectivity of the Internet has allowed geographically widely dispersed groups of people to work together by means of all kinds of **groupware**, that is, software for collaborative editing, teleconferencing, and so on, as is illustrated by multinational software-development companies that have outsourced much of their code production to Asia.

However, resource sharing in distributed systems is perhaps best illustrated by the success of file-sharing peer-to-peer networks like BitTorrent. These distributed systems make it extremely simple for users to share files across the Internet. Peer-to-peer networks are often associated with distribution of media files such as audio and video. In other cases, the technology is used for distributing large amounts of data, as in the case of software updates, backup services, and data synchronization across multiple servers.

Note 1.2 (More information: Sharing folders worldwide)

To illustrate where we stand when it comes to seamless integration of resource-sharing facilities in a networked environment, Web-based services are now deployed that allow a group of users to place files into a special shared folder that is

maintained by a third party somewhere on the Internet. Using special software, the shared folder is barely distinguishable from other folders on a user's computer. In effect, these services replace the use of a shared directory on a local distributed file system, making data available to users independent of the organization they belong to, and independent of where they are. The service is offered for different operating systems. Where exactly data are stored is completely hidden from the end user.

Making distribution transparent

An important goal of a distributed system is to hide the fact that its processes and resources are physically distributed across multiple computers possibly separated by large distances. In other words, it tries to make the distribution of processes and resources **transparent**, that is, invisible, to end users and applications.

Types of distribution transparency

The concept of transparency can be applied to several aspects of a distributed system, of which the most important ones are listed in Figure 1.2. We use the term *object* to mean either a process or a resource.

Transparency	Description
Access	Hide differences in data representation and how an object is accessed
Location	Hide where an object is located
Relocation	Hide that an object may be moved to another location while in use
Migration	Hide that an object may move to another location
Replication	Hide that an object is replicated
Concurrency	Hide that an object may be shared by several independent users
Failure	Hide the failure and recovery of an object

Figure 1.2: Different forms of transparency in a distributed system (see ISO [1995]). An object can be a resource or a process.

Access transparency deals with hiding differences in data representation and the way that objects can be accessed. At a basic level, we want to hide differences in machine architectures, but more important is that we reach agreement on how data is to be represented by different machines and operating systems. For example, a distributed system may have computer systems

that run different operating systems, each having their own file-naming conventions. Differences in naming conventions, differences in file operations, or differences in how low-level communication with other processes is to take place, are examples of access issues that should preferably be hidden from users and applications.

An important group of transparency types concerns the location of a process or resource. **Location transparency** refers to the fact that users cannot tell where an object is physically located in the system. Naming plays an important role in achieving location transparency. In particular, location transparency can often be achieved by assigning only logical names to resources, that is, names in which the location of a resource is not secretly encoded. An example of such a name is the **uniform resource locator** (URL) <http://www.prenhall.com/index.html>, which gives no clue about the actual location of Prentice Hall's main Web server. The URL also gives no clue as to whether the file `index.html` has always been at its current location or was recently moved there. For example, the entire site may have been moved from one data center to another, yet users should not notice. The latter is an example of **relocation transparency**, which is becoming increasingly important in the context of cloud computing to which we return later in this chapter.

Where relocation transparency refers to *being* moved by the distributed system, **migration transparency** is offered by a distributed system when it supports the mobility of processes and resources initiated by users, without affecting ongoing communication and operations. A typical example is communication between mobile phones: regardless whether two people are actually moving, mobile phones will allow them to continue their conversation. Other examples that come to mind include online tracking and tracing of goods as they are being transported from one place to another, and teleconferencing (partly) using devices that are equipped with mobile Internet.

As we shall see, replication plays an important role in distributed systems. For example, resources may be replicated to increase availability or to improve performance by placing a copy close to the place where it is accessed. **Replication transparency** deals with hiding the fact that several copies of a resource exist, or that several processes are operating in some form of lockstep mode so that one can take over when another fails. To hide replication from users, it is necessary that all replicas have the same name. Consequently, a system that supports replication transparency should generally support location transparency as well, because it would otherwise be impossible to refer to replicas at different locations.

We already mentioned that an important goal of distributed systems is to allow sharing of resources. In many cases, sharing resources is done in a cooperative way, as in the case of communication channels. However, there are also many examples of competitive sharing of resources. For example,

two independent users may each have stored their files on the same file server or may be accessing the same tables in a shared database. In such cases, it is important that each user does not notice that the other is making use of the same resource. This phenomenon is called **concurrency transparency**. An important issue is that concurrent access to a shared resource leaves that resource in a consistent state. Consistency can be achieved through locking mechanisms, by which users are, in turn, given exclusive access to the desired resource. A more refined mechanism is to make use of transactions, but these may be difficult to implement in a distributed system, notably when scalability is an issue.

Last, but certainly not least, it is important that a distributed system provides **failure transparency**. This means that a user or application does not notice that some piece of the system fails to work properly, and that the system subsequently (and automatically) recovers from that failure. Masking failures is one of the hardest issues in distributed systems and is even impossible when certain apparently realistic assumptions are made, as we will discuss in Chapter 8. The main difficulty in masking and transparently recovering from failures lies in the inability to distinguish between a dead process and a painfully slowly responding one. For example, when contacting a busy Web server, a browser will eventually time out and report that the Web page is unavailable. At that point, the user cannot tell whether the server is actually down or that the network is badly congested.

Degree of distribution transparency

Although distribution transparency is generally considered preferable for any distributed system, there are situations in which attempting to blindly hide all distribution aspects from users is not a good idea. A simple example is requesting your electronic newspaper to appear in your mailbox before 7 AM local time, as usual, while you are currently at the other end of the world living in a different time zone. Your morning paper will not be the morning paper you are used to.

Likewise, a wide-area distributed system that connects a process in San Francisco to a process in Amsterdam cannot be expected to hide the fact that Mother Nature will not allow it to send a message from one process to the other in less than approximately 35 milliseconds. Practice shows that it actually takes several hundred milliseconds using a computer network. Signal transmission is not only limited by the speed of light, but also by limited processing capacities and delays in the intermediate switches.

There is also a trade-off between a high degree of transparency and the performance of a system. For example, many Internet applications repeatedly try to contact a server before finally giving up. Consequently, attempting to mask a transient server failure before trying another one may slow down the

system as a whole. In such a case, it may have been better to give up earlier, or at least let the user cancel the attempts to make contact.

Another example is where we need to guarantee that several replicas, located on different continents, must be consistent all the time. In other words, if one copy is changed, that change should be propagated to all copies before allowing any other operation. It is clear that a single update operation may now even take seconds to complete, something that cannot be hidden from users.

Finally, there are situations in which it is not at all obvious that hiding distribution is a good idea. As distributed systems are expanding to devices that people carry around and where the very notion of location and context awareness is becoming increasingly important, it may be best to actually *expose* distribution rather than trying to hide it. An obvious example is making use of location-based services, which can often be found on mobile phones, such as finding the nearest Chinese take-away or checking whether any of your friends are nearby.

There are also other arguments against distribution transparency. Recognizing that full distribution transparency is simply impossible, we should ask ourselves whether it is even wise to *pretend* that we can achieve it. It may be much better to make distribution explicit so that the user and application developer are never tricked into believing that there is such a thing as transparency. The result will be that users will much better understand the (sometimes unexpected) behavior of a distributed system, and are thus much better prepared to deal with this behavior.

Note 1.3 (Discussion: Against distribution transparency)

Several researchers have argued that hiding distribution will only lead to further complicating the development of distributed systems, exactly for the reason that full distribution transparency can never be achieved. A popular technique for achieving access transparency is to extend procedure calls to remote servers. However, Waldo et al. [1997] already pointed out that attempting to hide distribution by means of such remote procedure calls can lead to poorly understood semantics, for the simple reason that a procedure call *does* change when executed over a faulty communication link.

As an alternative, various researchers and practitioners are now arguing for less transparency, for example, by more explicitly using message-style communication, or more explicitly posting requests to, and getting results from remote machines, as is done in the Web when fetching pages. Such solutions will be discussed in detail in the next chapter.

A somewhat radical standpoint is taken by Wams [2011] by stating that partial failures preclude relying on the successful execution of a remote service. If such reliability cannot be guaranteed, it is then best to always perform only local executions, leading to the **copy-before-use** principle. According to this principle, data can be accessed only after they have been transferred to the machine of the

process wanting that data. Moreover, modifying a data item should not be done. Instead, it can only be updated to a new version. It is not difficult to imagine that many other problems will surface. However, Wams shows that many existing applications can be retrofitted to this alternative approach without sacrificing functionality.

The conclusion is that aiming for distribution transparency may be a nice goal when designing and implementing distributed systems, but that it should be considered together with other issues such as performance and comprehensibility. The price for achieving full transparency may be surprisingly high.

Being open

Another important goal of distributed systems is openness. An **open distributed system** is essentially a system that offers components that can easily be used by, or integrated into other systems. At the same time, an open distributed system itself will often consist of components that originate from elsewhere.

Interoperability, composability, and extensibility

To be open means that components should adhere to standard rules that describe the syntax and semantics of what those components have to offer (i.e., which service they provide). A general approach is to define services through **interfaces** using an **Interface Definition Language (IDL)**. Interface definitions written in an IDL nearly always capture only the syntax of services. In other words, they specify precisely the names of the functions that are available together with types of the parameters, return values, possible exceptions that can be raised, and so on. The hard part is specifying precisely what those services do, that is, the semantics of interfaces. In practice, such specifications are given in an informal way by means of natural language.

If properly specified, an interface definition allows an arbitrary process that needs a certain interface, to talk to another process that provides that interface. It also allows two independent parties to build completely different implementations of those interfaces, leading to two separate components that operate in exactly the same way.

Proper specifications are complete and neutral. Complete means that everything that is necessary to make an implementation has indeed been specified. However, many interface definitions are not at all complete, so that it is necessary for a developer to add implementation-specific details. Just as important is the fact that specifications do not prescribe what an implementation should look like; they should be neutral.

As pointed out in Blair and Stefani [1998], completeness and neutrality are important for interoperability and portability. **Interoperability** characterizes the extent by which two implementations of systems or components from different manufacturers can co-exist and work together by merely relying on each other's services as specified by a common standard. **Portability** characterizes to what extent an application developed for a distributed system A can be executed, without modification, on a different distributed system B that implements the same interfaces as A.

Another important goal for an open distributed system is that it should be easy to configure the system out of different components (possibly from different developers). Also, it should be easy to add new components or replace existing ones without affecting those components that stay in place. In other words, an open distributed system should also be **extensible**. For example, in an extensible system, it should be relatively easy to add parts that run on a different operating system, or even to replace an entire file system.

Note 1.4 (Discussion: Open systems in practice)

Of course, what we have just described is an ideal situation. Practice shows that many distributed systems are not as open as we would like and that still a lot of effort is needed to put various bits and pieces together to make a distributed system. One way out of the lack of openness is to simply reveal all the gory details of a component and to provide developers with the actual source code. This approach is becoming increasingly popular, leading to so-called open source projects where large groups of people contribute to improving and debugging systems. Admittedly, this is as open as a system can get, but if it is the best way is questionable.

Separating policy from mechanism

To achieve flexibility in open distributed systems, it is crucial that the system be organized as a collection of relatively small and easily replaceable or adaptable components. This implies that we should provide definitions of not only the highest-level interfaces, that is, those seen by users and applications, but also definitions for interfaces to internal parts of the system and describe how those parts interact. This approach is relatively new. Many older and even contemporary systems are constructed using a monolithic approach in which components are only logically separated but implemented as one, huge program. This approach makes it hard to replace or adapt a component without affecting the entire system. Monolithic systems thus tend to be closed instead of open.

The need for changing a distributed system is often caused by a component that does not provide the optimal policy for a specific user or application. As an example, consider caching in Web browsers. There are many different parameters that need to be considered:

Storage: Where is data to be cached? Typically, there will be an in-memory cache next to storage on disk. In the latter case, the exact position in the local file system needs to be considered.

Exemption: When the cache fills up, which data is to be removed so that newly fetched pages can be stored?

Sharing: Does each browser make use of a private cache, or is a cache to be shared among browsers of different users?

Refreshing: When does a browser check if cached data is still up-to-date? Caches are most effective when a browser can return pages without having to contact the original Web site. However, this bears the risk of returning stale data. Note also that refresh rates are highly dependent on which data is actually cached: whereas timetables for trains hardly change, this is not the case for Web pages showing current highway-traffic conditions, or worse yet, stock prices.

What we need is a separation between policy and mechanism. In the case of Web caching, for example, a browser should ideally provide facilities for only storing documents and at the same time allow users to decide which documents are stored and for how long. In practice, this can be implemented by offering a rich set of parameters that the user can set (dynamically). When taking this a step further, a browser may even offer facilities for plugging in policies that a user has implemented as a separate component.

Note 1.5 (Discussion: Is a strict separation really what we need?)

In theory, strictly separating policies from mechanisms seems to be the way to go. However, there is an important trade-off to consider: the stricter the separation, the more we need to make sure that we offer the appropriate collection of mechanisms. In practice this means that a rich set of features is offered, in turn leading to many configuration parameters. As an example, the popular Firefox browser comes with a few hundred configuration parameters. Just imagine how the configuration space explodes when considering large distributed systems consisting of many components. In other words, strict separation of policies and mechanisms may lead to highly complex configuration problems.

One option to alleviate these problems is to provide reasonable defaults, and this is what often happens in practice. An alternative approach is one in which the system observes its own usage and dynamically changes parameter settings. This leads to what are known as **self-configurable systems**. Nevertheless, the fact alone that many mechanisms need to be offered in order to support a wide range of policies often makes coding distributed systems very complicated. Hard coding policies into a distributed system may reduce complexity considerably, but at the price of less flexibility.

Finding the right balance in separating policies from mechanisms is one of the reasons why designing a distributed system is often more an art than a science.

Being scalable

For many of us, worldwide connectivity through the Internet is as common as being able to send a postcard to anyone anywhere around the world. Moreover, where until recently we were used to having relatively powerful desktop computers for office applications and storage, we are now witnessing that such applications and services are being placed in what has been coined “the cloud,” in turn leading to an increase of much smaller networked devices such as tablet computers. With this in mind, scalability has become one of the most important design goals for developers of distributed systems.

Scalability dimensions

Scalability of a system can be measured along at least three different dimensions (see [Neuman, 1994]):

Size scalability: A system can be scalable with respect to its size, meaning that we can easily add more users and resources to the system without any noticeable loss of performance.

Geographical scalability: A geographically scalable system is one in which the users and resources may lie far apart, but the fact that communication delays may be significant is hardly noticed.

Administrative scalability: An administratively scalable system is one that can still be easily managed even if it spans many independent administrative organizations.

Let us take a closer look at each of these three scalability dimensions.

Size scalability. When a system needs to scale, very different types of problems need to be solved. Let us first consider scaling with respect to size. If more users or resources need to be supported, we are often confronted with the limitations of centralized services, although often for very different reasons. For example, many services are centralized in the sense that they are implemented by means of a single **server** running on a specific machine in the distributed system. In a more modern setting, we may have a group of collaborating servers co-located on a cluster of tightly coupled machines physically placed at the same location. The problem with this scheme is obvious: the server, or group of servers, can simply become a bottleneck when it needs to process an increasing number of requests. To illustrate how this can happen, let us assume that a service is implemented on a single machine. In that case there are essentially three root causes for becoming a bottleneck:

- The computational capacity, limited by the CPUs
- The storage capacity, including the I/O transfer rate

- The network between the user and the centralized service

Let us first consider the computational capacity. Just imagine a service for computing optimal routes taking real-time traffic information into account. It is not difficult to imagine that this may be primarily a compute-bound service requiring several (tens of) seconds to complete a request. If there is only a single machine available, then even a modern high-end system will eventually run into problems if the number of requests increases beyond a certain point.

Likewise, but for different reasons, we will run into problems when having a service that is mainly I/O bound. A typical example is a poorly designed centralized search engine. The problem with content-based search queries is that we essentially need to match a query against an entire data set. Even with advanced indexing techniques, we may still face the problem of having to process a huge amount of data exceeding the main-memory capacity of the machine running the service. As a consequence, much of the processing time will be determined by the relatively slow disk accesses and transfer of data between disk and main memory. Simply adding more or higher-speed disks will prove not to be a sustainable solution as the number of requests continues to increase.

Finally, the network between the user and the service may also be the cause of poor scalability. Just imagine a video-on-demand service that needs to stream high-quality video to multiple users. A video stream can easily require a bandwidth of 8 to 10 Mbps, meaning that if a service sets up point-to-point connections with its customers, it may soon hit the limits of the network capacity of its own outgoing transmission lines.

There are several solutions to attack size scalability which we discuss below after having looked into geographical and administrative scalability.

Note 1.6 (Advanced: Analyzing service capacity)

Size scalability problems for centralized services can be formally analyzed using queuing theory and making a few simplifying assumptions. At a conceptual level, a centralized service can be modeled as the simple queuing system shown in Figure 1.3: requests are submitted to the service where they are queued until further notice. As soon as the process can handle a next request, it fetches it from the queue, does its work, and produces a response. We largely follow Menasce and Almeida [2002] in explaining the performance of a centralized service.

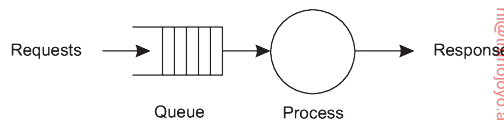


Figure 1.3: A simple model of a service as a queuing system.

In many cases, we may assume that the queue has an infinite capacity, meaning that there is no restriction on the number of requests that can be accepted for

further processing. Strictly speaking, this means that the arrival rate of requests is not influenced by what is currently in the queue or being processed. Assuming that the arrival rate of requests is λ requests per second, and that the processing capacity of the service is μ requests per second, one can compute that the fraction of time p_k that there are k requests in the system is equal to:

$$p_k = \left(1 - \frac{\lambda}{\mu}\right) \left(\frac{\lambda}{\mu}\right)^k$$

If we define the **utilization** U of a service as the fraction of time that it is busy, then clearly,

$$U = \sum_{k>0} p_k = 1 - p_0 = \frac{\lambda}{\mu} \Rightarrow p_k = (1 - U)U^k$$

We can then compute the average number $\bar{N}$ of requests in the system as

$$\bar{N} = \sum_{k \geq 0} k \cdot p_k = \sum_{k \geq 0} k \cdot (1 - U)U^k = (1 - U) \sum_{k \geq 0} k \cdot U^k = \frac{(1 - U)U}{(1 - U)^2} = \frac{U}{1 - U}.$$

What we are really interested in, is the response time R : how long does it take before the service to process a request, including the time spent in the queue. To that end, we need the average throughput X . Considering that the service is “busy” when at least one request is being processed, and that this then happens with a throughput of μ requests per second, and during a fraction U of the total time, we have:

$$X = \underbrace{U \cdot \mu}_{\text{server at work}} + \underbrace{(1 - U) \cdot 0}_{\text{server idle}} = \frac{\lambda}{\mu} \cdot \mu = \lambda$$

Using Little’s formula [Trivedi, 2002], we can then derive the response time as

$$R = \frac{\bar{N}}{X} = \frac{S}{1 - U} \Rightarrow \frac{R}{S} = \frac{1}{1 - U}$$

where $S = \frac{1}{\mu}$, the actual service time. Note that if U is very small, the response-to-service time ratio is close to 1, meaning that a request is virtually instantly processed, and at the maximum speed possible. However, as soon as the utilization comes closer to 1, we see that the response-to-server time ratio quickly increases to very high values, effectively meaning that the system is coming close to a grinding halt. This is where we see scalability problems emerge. From this simple model, we can see that the only solution is bringing down the service time S . We leave it as an exercise to the reader to explore how S may be decreased.

Geographical scalability. Geographical scalability has its own problems. One of the main reasons why it is still difficult to scale existing distributed systems that were designed for local-area networks is that many of them are based on **synchronous communication**. In this form of communication, a

party requesting service, generally referred to as a **client**, blocks until a reply is sent back from the **server** implementing the service. More specifically, we often see a communication pattern consisting of many client-server interactions as may be the case with database transactions. This approach generally works fine in LANs where communication between two machines is often at worst a few hundred microseconds. However, in a wide-area system, we need to take into account that interprocess communication may be hundreds of milliseconds, three orders of magnitude slower. Building applications using synchronous communication in wide-area systems requires a great deal of care (and not just a little patience), notably with a rich interaction pattern between client and server.

Another problem that hinders geographical scalability is that communication in wide-area networks is inherently much less reliable than in local-area networks. In addition, we also need to deal with limited bandwidth. The effect is that solutions developed for local-area networks cannot always be easily ported to a wide-area system. A typical example is streaming video. In a home network, even when having only wireless links, ensuring a stable, fast stream of high-quality video frames from a media server to a display is quite simple. Simply placing that same server far away and using a standard TCP connection to the display will surely fail: bandwidth limitations will instantly surface, but also maintaining the same level of reliability can easily cause headaches.

Yet another issue that pops up when components lie far apart is the fact that wide-area systems generally have only very limited facilities for multipoint communication. In contrast, local-area networks often support efficient broadcasting mechanisms. Such mechanisms have proven to be extremely useful for discovering components and services, which is essential from a management point of view. In wide-area systems, we need to develop separate services, such as naming and directory services to which queries can be sent. These support services, in turn, need to be scalable as well and in many cases no obvious solutions exist as we will encounter in later chapters.

Administrative scalability. Finally, a difficult, and in many cases open, question is how to scale a distributed system across multiple, independent administrative domains. A major problem that needs to be solved is that of conflicting policies with respect to resource usage (and payment), management, and security.

To illustrate, for many years scientists have been looking for solutions to share their (often expensive) equipment in what is known as a **computational grid**. In these grids, a global distributed system is constructed as a federation of local distributed systems, allowing a program running on a computer at organization A to directly access resources at organization B.

For example, many components of a distributed system that reside within

a single domain can often be trusted by users that operate within that same domain. In such cases, system administration may have tested and certified applications, and may have taken special measures to ensure that such components cannot be tampered with. In essence, the users trust their system administrators. However, this trust does not expand naturally across domain boundaries.

Note 1.7 (Example: A modern radio telescope)

As an example, consider developing a modern radio telescope, such as the Pierre Auger Observatory [Abraham et al., 2004]. The final system can be considered as a federated distributed system:

- The radio telescope itself may be a wireless distributed system developed as a grid of a few thousand sensor nodes, each collecting radio signals and collaborating with neighboring nodes to filter out relevant events. The nodes dynamically maintain a sink tree by which selected events are routed to a central point for further analysis.
- The central point needs to be a reasonably powerful system, capable of storing and processing the events sent to it by the sensor nodes. This system is necessarily placed in proximity of the sensor nodes, but is otherwise to be considered to operate independently. Depending on its functionality, it may operate as a small local distributed system. In particular, it stores all recorded events and offers access to remote systems owned by partners in the consortium.
- Most partners have local distributed systems (often in the form of a cluster of computers) that they use to further process the data collected by the telescope. In this case, the local systems directly access the central point at the telescope using a standard communication protocol. Naturally, many results produced within the consortium are made available to each partner.

It is thus seen that the complete system will cross boundaries of several administrative domains, and that special measures are needed to ensure that data that is supposed to be accessible only to (specific) consortium partners cannot be disclosed to unauthorized parties. How to achieve administrative scalability is not obvious.

If a distributed system expands to another domain, two types of security measures need to be taken. First, the distributed system has to protect itself against malicious attacks from the new domain. For example, users from the new domain may have only read access to the file system in its original domain. Likewise, facilities such as expensive image setters or high-performance computers may not be made available to unauthorized users. Second, the new domain has to protect itself against malicious attacks from the distributed system. A typical example is that of downloading programs such as applets in Web browsers. Basically, the new domain does not know

what to expect from such foreign code. The problem, as we shall see in Chapter 9, is how to enforce those limitations.

As a counterexample of distributed systems spanning multiple administrative domains that apparently *do not* suffer from administrative scalability problems, consider modern file-sharing peer-to-peer networks. In these cases, end users simply install a program implementing distributed search and download functions and within minutes can start downloading files. Other examples include peer-to-peer applications for telephony over the Internet such as Skype [Baset and Schulzrinne, 2006], and peer-assisted audio-streaming applications such as Spotify [Kreitz and Niemelä, 2010]. What these distributed systems have in common is that *end users*, and not administrative entities, collaborate to keep the system up and running. At best, underlying administrative organizations such as **Internet Service Providers (ISPs)** can police the network traffic that these peer-to-peer systems cause, but so far such efforts have not been very effective.

Scaling techniques

Having discussed some of the scalability problems brings us to the question of how those problems can generally be solved. In most cases, scalability problems in distributed systems appear as performance problems caused by limited capacity of servers and network. Simply improving their capacity (e.g., by increasing memory, upgrading CPUs, or replacing network modules) is often a solution, referred to as **scaling up**. When it comes to **scaling out**, that is, expanding the distributed system by essentially deploying more machines, there are basically only three techniques we can apply: hiding communication latencies, distribution of work, and replication (see also Neuman [1994]).

Hiding communication latencies. Hiding communication latencies is applicable in the case of geographical scalability. The basic idea is simple: try to avoid waiting for responses to remote-service requests as much as possible. For example, when a service has been requested at a remote machine, an alternative to waiting for a reply from the server is to do other useful work at the requester's side. Essentially, this means constructing the requesting application in such a way that it uses only **asynchronous communication**. When a reply comes in, the application is interrupted and a special handler is called to complete the previously issued request. Asynchronous communication can often be used in batch-processing systems and parallel applications in which independent tasks can be scheduled for execution while another task is waiting for communication to complete. Alternatively, a new thread of control can be started to perform the request. Although it blocks waiting for the reply, other threads in the process can continue.

However, there are many applications that cannot make effective use of asynchronous communication. For example, in interactive applications when

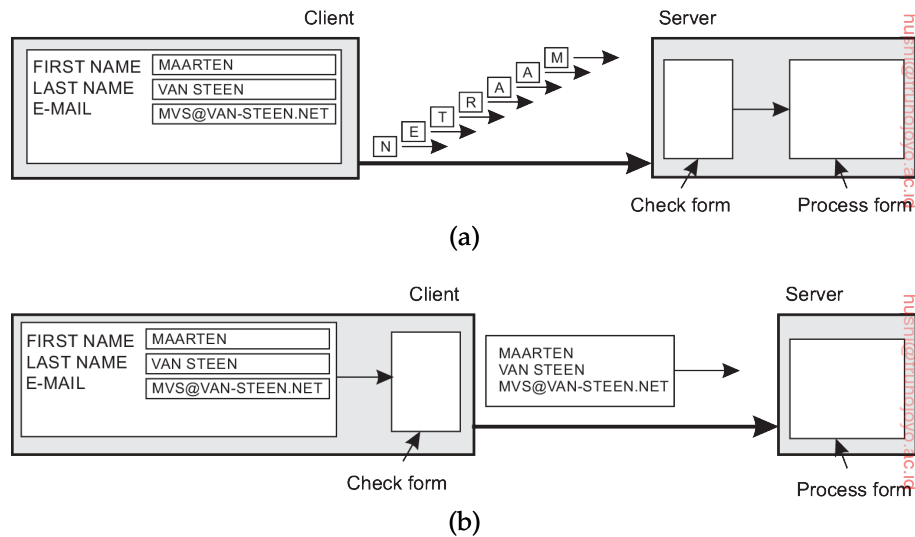


Figure 1.4: The difference between letting (a) a server or (b) a client check forms as they are being filled.

a user sends a request he will generally have nothing better to do than to wait for the answer. In such cases, a much better solution is to reduce the overall communication, for example, by moving part of the computation that is normally done at the server to the client process requesting the service. A typical case where this approach works is accessing databases using forms. Filling in forms can be done by sending a separate message for each field and waiting for an acknowledgment from the server, as shown in Figure 1.4(a). For example, the server may check for syntactic errors before accepting an entry. A much better solution is to ship the code for filling in the form, and possibly checking the entries, to the client, and have the client return a completed form, as shown in Figure 1.4(b). This approach of shipping code is widely supported by the Web by means of Java applets and Javascript.

Partitioning and distribution. Another important scaling technique is **partitioning and distribution**, which involves taking a component, splitting it into smaller parts, and subsequently spreading those parts across the system. A good example of partitioning and distribution is the Internet Domain Name System (DNS). The DNS name space is hierarchically organized into a tree of **domains**, which are divided into nonoverlapping **zones**, as shown for the original DNS in Figure 1.5. The names in each zone are handled by a single name server. Without going into too many details now (we return to DNS extensively in Chapter 5), one can think of each path name being the name of a host in the Internet, and is thus associated with a network address of that

host. Basically, resolving a name means returning the network address of the associated host. Consider, for example, the name `flits.cs.vu.nl`. To resolve this name, it is first passed to the server of zone Z1 (see Figure 1.5) which returns the address of the server for zone Z2, to which the rest of name, `flits.cs.vu`, can be handed. The server for Z2 will return the address of the server for zone Z3, which is capable of handling the last part of the name and will return the address of the associated host.

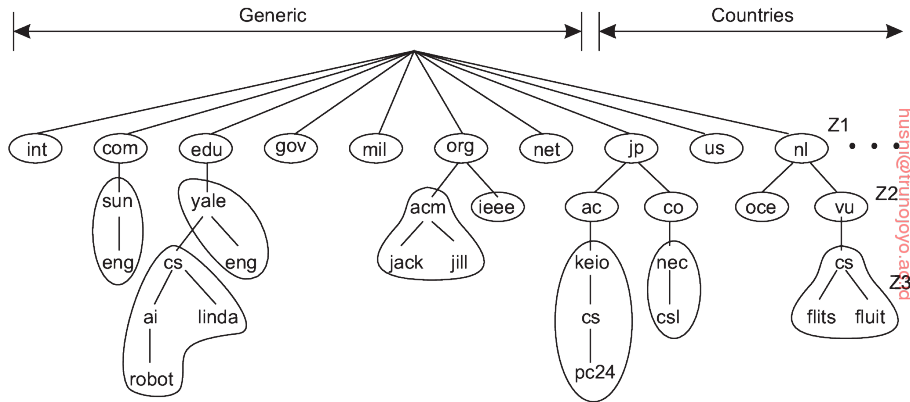


Figure 1.5: An example of dividing the (original) DNS name space into zones.

This examples illustrates how the *naming service*, as provided by DNS, is distributed across several machines, thus avoiding that a single server has to deal with all requests for name resolution.

As another example, consider the World Wide Web. To most users, the Web appears to be an enormous document-based information system in which each document has its own unique name in the form of a URL. Conceptually, it may even appear as if there is only a single server. However, the Web is physically partitioned and distributed across *a few hundred million* servers, each handling a number of Web documents. The name of the server handling a document is encoded into that document's URL. It is only because of this distribution of documents that the Web has been capable of scaling to its current size.

Replication. Considering that scalability problems often appear in the form of performance degradation, it is generally a good idea to actually **replicate** components across a distributed system. Replication not only increases availability, but also helps to balance the load between components leading to better performance. Also, in geographically widely dispersed systems, having a copy nearby can hide much of the communication latency problems mentioned before.

Caching is a special form of replication, although the distinction between the two is often hard to make or even artificial. As in the case of replication,

caching results in making a copy of a resource, generally in the proximity of the client accessing that resource. However, in contrast to replication, caching is a decision made by the client of a resource and not by the owner of a resource.

There is one serious drawback to caching and replication that may adversely affect scalability. Because we now have multiple copies of a resource, modifying one copy makes that copy different from the others. Consequently, caching and replication leads to **consistency** problems.

To what extent inconsistencies can be tolerated depends highly on the usage of a resource. For example, many Web users find it acceptable that their browser returns a cached document of which the validity has not been checked for the last few minutes. However, there are also many cases in which strong consistency guarantees need to be met, such as in the case of electronic stock exchanges and auctions. The problem with strong consistency is that an update must be immediately propagated to all other copies. Moreover, if two updates happen concurrently, it is often also required that updates are processed in the same order everywhere, introducing an additional global ordering problem. To further aggravate problems, combining consistency with other desirable properties such as availability may simply be impossible, as we discuss in Chapter 8.

Replication therefore often requires some global synchronization mechanism. Unfortunately, such mechanisms are extremely hard or even impossible to implement in a scalable way, if alone because network latencies have a natural lower bound. Consequently, scaling by replication may introduce other, inherently nonscalable solutions. We return to replication and consistency extensively in Chapter 7.

Discussion. When considering these scaling techniques, one could argue that size scalability is the least problematic from a technical point of view. In many cases, increasing the capacity of a machine will save the day, although perhaps there is a high monetary cost to pay. Geographical scalability is a much tougher problem as network latencies are naturally bound from below. As a consequence, we may be forced to copy data to locations close to where clients are, leading to problems of maintaining copies consistent. Practice shows that combining distribution, replication, and caching techniques with different forms of consistency generally leads to acceptable solutions. Finally, administrative scalability seems to be the most difficult problem to solve, partly because we need to deal with nontechnical issues, such as politics of organizations and human collaboration. The introduction and now widespread use of peer-to-peer technology has successfully demonstrated what can be achieved if end users are put in control [Lua et al., 2005; Oram, 2001]. However, peer-to-peer networks are obviously not the universal solution to all administrative scalability problems.

Pitfalls

It should be clear by now that developing a distributed system is a formidable task. As we will see many times throughout this book, there are so many issues to consider at the same time that it seems that only complexity can be the result. Nevertheless, by following a number of design principles, distributed systems can be developed that strongly adhere to the goals we set out in this chapter.

Distributed systems differ from traditional software because components are dispersed across a network. Not taking this dispersion into account during design time is what makes so many systems needlessly complex and results in flaws that need to be patched later on. Peter Deutsch, at the time working at Sun Microsystems, formulated these flaws as the following false assumptions that everyone makes when developing a distributed application for the first time:

- The network is reliable
- The network is secure
- The network is homogeneous
- The topology does not change
- Latency is zero
- Bandwidth is infinite
- Transport cost is zero
- There is one administrator

Note how these assumptions relate to properties that are unique to distributed systems: reliability, security, heterogeneity, and topology of the network; latency and bandwidth; transport costs; and finally administrative domains. When developing nondistributed applications, most of these issues will most likely not show up.

Most of the principles we discuss in this book relate immediately to these assumptions. In all cases, we will be discussing solutions to problems that are caused by the fact that one or more assumptions are false. For example, reliable networks simply do not exist and lead to the impossibility of achieving failure transparency. We devote an entire chapter to deal with the fact that networked communication is inherently insecure. We have already argued that distributed systems need to be open and take heterogeneity into account. Likewise, when discussing replication for solving scalability problems, we are essentially tackling latency and bandwidth problems. We will also touch upon management issues at various points throughout this book.

1.3 Types of distributed systems

Before starting to discuss the principles of distributed systems, let us first take a closer look at the various types of distributed systems. We make a